

```
// Data definition list
ArrayList<String> oDDLList = new ArrayList<String>();
oDDLList.add("CREATE TABLE countries (" +
    " country_id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL," +
    " country_name TEXT UNIQUE NOT NULL," +
    " country_continent TEXT NOT NULL);");
oDB = new MvSQLiteDB("/sdcard/countries.sq3", oDDLList);
// Requires write permission in app manifest XML
if (oDB.mbIsAvailable) {
    oink.showMessage("DB success");
} else {
    oink.showMessage("DB failure");
}
```

If the database file or the specified tables already exist, the constructor will not fail. Without a worry, you can just move along to the next steps. Please note that, with SQLite, you do not need to create an AUTOINCREMENT column. The built-in ROWID column might work just as well.

Using an existing SQLite database

What if you need to use a SQLite database file that already exists? Just specify the path in the constructor and forget the table definition statements.

```
oDB = new MvSQLiteDB("/mnt/sdcard/countries.sq3");
```

What if you are paranoid and need to check whether the tables are actually there? Well, then check the *sql_master* table that SQLite automatically creates to store your database schema.

```
oDB = new MvSQLiteDB("/sdcard/countries.sq3");
MvException oCheckTable =
oDB.getSingleResultForSelectSQL(
    "SELECT COUNT(name) " +
    " FROM sqlite_master " +
    " WHERE TYPE='table' AND name IN ('COUNTRIES');",
    true); // returns a number
if (oCheckTable.mbSuccess) {
    if ((Long) oCheckTable.moResult == 1) { // number of tables
        oink.showMessage("Yes, countries table is present");
    } else {
        oink.showMessage("No, countries table is not present");
    }
}
```

AndroidWithoutStupid uses its *MvException* class to provide a universal object-type return variable (*MvException.moResult*) and any exception (*MvException.mException*) encountered along the way. This helps in avoiding unnecessary nesting and indentation with multiple try-catch blocks. You can rely on simpler C-style true-or-false evaluations and not worry about exceptions. This makes the code compact and easier to read. Of course, you

need to fix exceptions during development.

The exceptions are usually caused by mistakes in the SQL statements or by logical errors related to database constraints.

Adding records

AndroidWithoutStupid doesn't mess with ContentValues objects. It is SQL statements all the way.

```
oDB = new MvSQLiteDB("/sdcard/countries.sq3");

MvException oAddRecord;
oAddRecord = oDB.executeSQL(
    "INSERT INTO COUNTRIES (COUNTRY_NAME, COUNTRY_CONTINENT) " +
    " VALUES ('India', 'Asia');");
if (oAddRecord.mbSuccess) {
    oink.showMessage("India added");
} else {
    oink.showMessage("India was not added\n" +
        oAddRecord.mException.getMessage());
}
```

Like most methods in *MvSQLiteDB*, the *executeSQL()* method automatically opens and closes the database. If you are adding several records one after another, if you choose not to auto-close the DB, remember to call the *closeDB()* method at the end. Else, your app will be killed.

```
oDB = new MvSQLiteDB("/sdcard/countries.sq3");
oDB.executeSQL(
    "INSERT INTO countries (country_name, country_continent) " +
    "VALUES ('USA', 'North America');",
    false);
oDB.executeSQL(
    "INSERT INTO countries (country_name, country_continent) " +
    " VALUES ('UK', 'Europe');",
    false);
oDB.executeSQL(
    "INSERT INTO countries (country_name, country_continent) " +
    " VALUES ('Russia', 'Europe');",
    false);
oDB.executeSQL(
    "INSERT INTO countries (country_name, country_continent) " +
    " VALUES ('Egypt', 'Africa');",
    false);
// do not forget this line if false is used with executeSQL()
oDB.closeDB();
```

Deleting and updating records

The *executeSQL()* method can also be used for any other SQL statements that do not return results, such as those that delete and update records.

```
MvException oDelete, oUpdate;
```